Name: Kedar Somnath Dhotre

PRN: 23510090

Experiment No. 7

Title: MPI Collective Communication and Parallel Matrix Computations

1. Aim

To understand and implement MPI collective communication operations and apply them to parallel matrix computations using distributed-memory programming.

You will:

- Understand collective communication in MPI.
- Implement MPI_Bcast(), MPI_Scatter(), MPI_Gather(), and MPI_Reduce().
- Implement parallel Matrix–Vector Multiplication using MPI.
- Implement parallel Matrix–Matrix Multiplication using MPI.
- Apply different data decomposition strategies.
- Measure and analyze communication and computation performance.
- Understand the effect of the number of processes and decomposition strategy on performance.

2. Why Are We Learning This?

In distributed-memory HPC systems, data is divided among multiple processes. Unlike OpenMP, where threads share a common memory, MPI processes need to exchange data explicitly.

For example, consider a large matrix:

[C = A* B]

If the matrix is too large for a single machine or the computation is time-consuming, different portions of the matrix can be assigned to different processes.

MPI collective operations provide efficient mechanisms to distribute and combine this data.

These concepts are fundamental to applications such as:

- Scientific and engineering simulations
- Computational fluid dynamics
- Weather and climate modelling

- Image and video processing
- Machine learning and AI
- Large-scale numerical linear algebra
- Distributed data analytics

3. HPC in Practice

Consider a weather simulation working with a large numerical grid.

The complete grid cannot efficiently be processed by one process. The grid can be divided among multiple processes:

Process 0 → Region 1
Process 1 → Region 2
Process 2 → Region 3
Process 3 → Region 4 MPI

can then be used to:

1. Distribute input data.
2. Perform computation on local data.
3. Exchange required information.
4. Collect the final results.

This pattern of:

Distribute → Compute → Communicate → Combine

is one of the fundamental patterns in HPC programming.

4. Learning Focus

Concept	MPI Function / Technique
Broadcast data	MPI_Bcast()
Distribute data	MPI_Scatter()
Collect data	MPI_Gather()
Combine results	MPI_Reduce()
Synchronization	MPI_Barrier()
Matrix–Vector Multiplication	Row-wise decomposition
Matrix–Matrix Multiplication	Row-wise / block decomposition

Performance	Time, speedup, efficiency

Concept	MPI Function / Technique
Advanced concept	MPI_Scatterv()

5. Pre-Lab Activity

Before performing the assignment, answer the following:

1. What is collective communication in MPI?

   Collective communication is communication where all processes in a communicator participate in the same communication operation.
   Examples: MPI_Bcast(), MPI_Scatter(), MPI_Gather(), MPI_Reduce(), MPI_Barrier().
2. How is collective communication different from point-to-point communication?
   - Collective: involves multiple/all processes together.
   - Point-to-point: communication occurs between two specific processes, using functions like MPI_Send() and MPI_Recv().
3. What is the purpose of MPI_Bcast()?

   MPI_Bcast() broadcasts the same data from one root process to all other processes.
4. What is the difference between MPI_Scatter() and MPI_Gather()?
   - `MPI_Scatter()` → one process distributes data among all processes.
   - `MPI_Gather()` → all processes send their data to one process.
5. What does MPI_Reduce() do?

   MPI_Reduce() combines data from all processes using an operation such as sum, maximum, minimum, or product, and stores the result at the root process.
6. Why is MPI_Barrier() used?

   MPI_Barrier() synchronizes all processes. No process can continue beyond the barrier until all processes have reached it.
7. What is data decomposition?

   Data decomposition means dividing a large dataset or problem into smaller parts so that different processes can work on them simultaneously.
8. Why is matrix computation suitable for parallel processing?

   Matrix computations often involve independent operations on different rows or elements, so multiple processes can calculate different parts simultaneously, reducing execution time.
9. What is row-wise decomposition?

   Row-wise decomposition means dividing the rows of a matrix among different processes. Each process performs computation on its assigned rows.

10. What happens if the number of matrix rows is not evenly divisible by the number of processes?

    The rows cannot be divided equally. Some processes receive one extra row, while others receive fewer rows. This can be handled using unequal distribution, such as MPI_Scatterv(), to specify different numbers of rows for different processes.

Section A – Basics: MPI Collective Communication

Task 1: Broadcast Using MPI_Bcast()

Write an MPI program in which:

- Process 0 initializes an integer value.
- Process 0 broadcasts the value to all other processes using MPI_Bcast().
- Every process displays the received value and its rank.

Suggested execution

Run the program using:

- 2 processes
- 4 processes
- 8 processes

Observe

Create a table:

Number of Processes	Process Ranks	Received Value
2	0, 1	100
4	0, 1, 2, 3	100
8	0, 1, 2, 3, 4, 5, 6, 7	100

Think

Why is broadcasting more appropriate than performing multiple individual MPI_Send() operations?
MPI_Bcast() is more appropriate because it efficiently sends the same data from one root process to all processes using a single collective communication operation, whereas multiple MPI_Send() operations require separate sends to each process.

Task 2: Scatter and Gather

Create an integer array of size 16 at Process 0.

Use:

MPI_Scatter()

to distribute the array among all processes.

Each process should:

1. Receive its portion.
2. Calculate the local sum.
3. Send the local result back using MPI_Gather().

Display the local sum calculated by eachprocess and the final result at Process 0.

Example

For:

Array = [1,2,3,...,16]

with 4 processes:

Process 0 → [1,2,3,4]
Process 1 → [5,6,7,8]
Process 2 → [9,10,11,12]
Process 3 → [13,14,15,16]

Task 3: Global Reduction Using MPI_Reduce()

Modify Task 2 to use:

MPI_Reduce()

to calculate the global sum.

Also calculate:

- Global maximum
- Global minimum

using appropriate MPI reduction operations.

Compare the use of MPI_Gather() and MPI_Reduce().

MPI_Gather()	MPI_Reduce()
Collects data from all processes at the root process.	Combines data from all processes into one final result at the root process.
Root receives all individual values.	Root receives only the final aggregate value.
The root may need to perform further calculations.	MPI performs the calculation using operations like MPI_SUM, MPI_MAX, MPI_MIN.
Example: [10, 26, 42, 58]	Example: 10 + 26 + 42 + 58 = 136
Used when individual results are needed.	Used when only the combined result is needed.

Discussion

Why is MPI_Reduce() preferable when only the final aggregate value is required?

MPI_Reduce() is preferable because it directly combines the values from all processes and gives only the final result to the root process. It avoids collecting all individual values and performing the calculation separately at the root. Therefore, it is simpler and can reduce unnecessary data handling.

Section B – Moderate Application: Matrix Computations

Task 4: MPI Matrix–Vector Multiplication

Implement:

[Y = A X]

where:

- (A) is an (N N) matrix.
- (X) is an (N) vector.
- (Y) is an (N) result vector.

Step 1 – Sequential Version

Implement the matrix-vector multiplication sequentially.

Measure its execution time.

Step 2 – MPI Version

Implement the MPI version using row-wise decomposition.

Vector X → Broadcast to all processes

Use:

- MPI_Scatter() to distribute matrix rows.
- MPI_Bcast() to distribute the vector.
- Local computation at each process.
- MPI_Gather() to collect the result.

Performance Analysis

Test with different matrix sizes, for example:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

and different numbers of processes:

- 2
- 4
- 8

Record:

Matrix Size	Processes	Sequential Time (s)	MPI Time (s)	Speedup	Efficiency
500	2	0.001358	0.001674	0.8112	40.56%
500	4	0.001358	0.002051	0.6621	16.55%
500	8	0.001358	0.002923	0.4646	5.81%
1000	2	0.006474	0.006103	1.0608	53.04%
1000	4	0.006474	0.008340	0.7763	19.41%
1000	8	0.006474	0.012365	0.5236	6.54%
2000	2	0.008145	0.024747	0.3291	16.46%
2000	4	0.008145	0.030066	0.2709	6.77%

2000	8	0.008145	0.038596	0.2110	2.64%

Task 5: MPI Matrix–Matrix Multiplication

Implement:

[C = A * B]

for two square matrices (A) and (B).

Sequential Version

First implement the sequential algorithm.

MPI Version

Implement a parallel version using row-wise decomposition.

Suggested approach:

1. Process 0 initializes matrices (A) and (B).
2. Distribute rows of (A) using MPI_Scatter().
3. Broadcast matrix (B) using MPI_Bcast().
4. Each process computes its assigned rows of (C).
5. Collect the rows of (C) using MPI_Gather().

Section C – Challenge: Decomposition and Optimization

Task 6: Compare Different Matrix Decomposition Strategies

Modify the MPI matrix-matrix multiplication program to investigate different decomposition strategies.

Study:

1. Row-wise decomposition

Each process receives a set of rows of matrix (A).

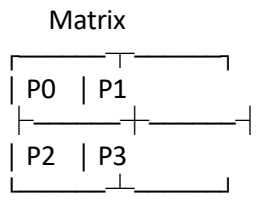
2. Column-wise decomposition

Each process is responsible for a set of columns of matrix (B).

3. Block decomposition

Divide the matrix into smaller blocks.

For example:



Investigate how the decomposition affects:

- Amount of data communicated.
- Computation performed by each process.
- Memory usage.
- Load balancing.
- Execution time.
- Scalability.

Factor	Row-wise	Column-wise	Block
Data communicated	Rows of A are scattered; B is broadcast to all processes; C rows are gathered	A is broadcast; columns of B are distributed; C columns are gathered	Blocks of A and B are distributed and block results require communication between process groups
Computation per process	Each process computes its assigned rows of C	Each process computes its assigned columns of C	Each process computes one block of C and must combine results from multiple block multiplications
Memory usage	Each process stores its local rows of A and the complete B	Each process stores complete A and its local columns of B	Each process mainly stores local A, B and C blocks, so memory can be distributed more efficiently
Load balancing	Good when rows are distributed equally	Good when columns are distributed equally	Good when blocks are evenly distributed; requires suitable process grid
Execution time	**0.158808 s**	**0.140988 s**	**0.074886 s***
Scalability	Works well initially, but broadcasting B becomes costly with more processes	Similar communication overhead due to broadcasting A	Better potential scalability because computation and data can be distributed in 2D

Task 7: Non-Uniform Data Distribution Using MPI_Scatterv()

Consider a situation where the matrix cannot be divided equally among the processes.

For example:

N = 1 0
Number of processes = 3

The rows cannot be divided equally.

Investigate and implement matrix-vector multiplication using:

MPI_Scatterv()

instead of MPI_Scatter().

The distribution could be:

P0 → 4 rows
P1 → 3 rows
P2 → 3 rows

Analyze

Compare:

MPI_Scatter()
vs.
MPI_Scatterv()

in terms of:

- Data distribution
- Flexibility
- Programming complexity
- Applicability to real HPC problems

Parameter	MPI_Scatter()	MPI_Scatterv()
Data distribution	Distributes the same number of elements to every process.	Can distribute different numbers of elements to different processes.
Example for N=10, 3 processes	Cannot naturally distribute 4, 3, 3 rows.	Successfully distributes P0 → 4, P1 → 3, P2 → 3 rows.
Flexibility	Less flexible; suitable when data is evenly divisible.	More flexible; suitable for uneven or irregular data.
Programming complexity	Simple. Only the count of elements is specified.	Slightly more complex because sendcounts[] and displs[] are required.
Applicability to HPC	Suitable for regular workloads with equal-sized data.	Suitable for irregular workloads such as large matrices, sparse data, graph processing, and uneven workloads.

6. Performance Analysis

For your MPI programs, measure:

Execution Time

Speedup

Efficiency

where (p) is the number of processes.

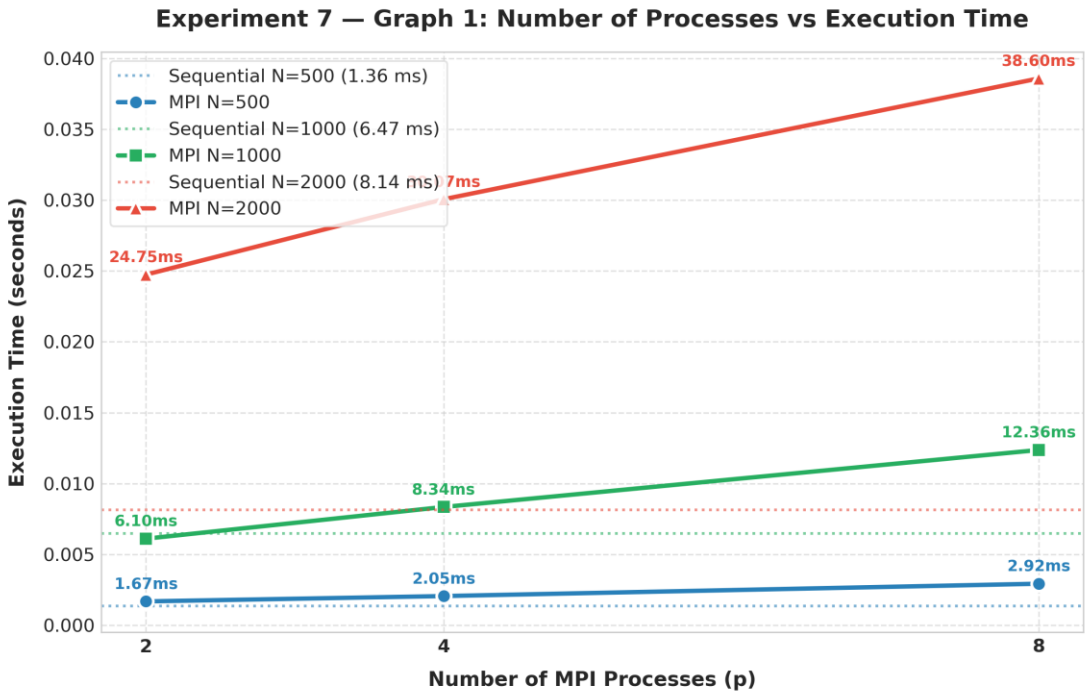
Also investigate:

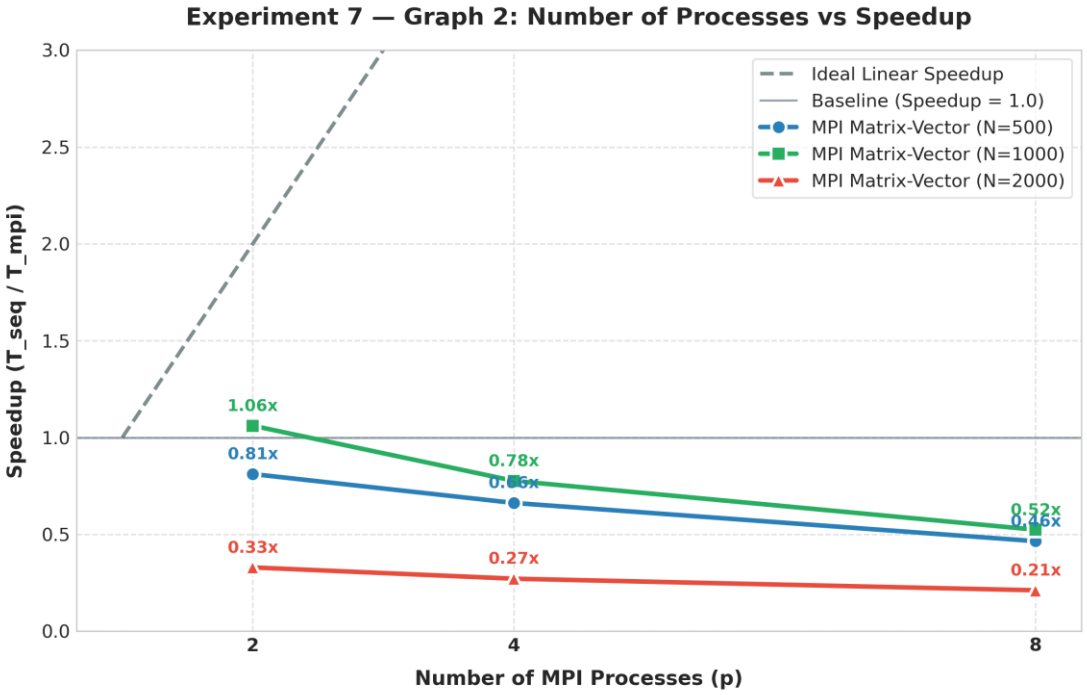
- Communication overhead
- Computation time
- Load imbalance
- Scalability
- Effect of matrix size
- Effect of number of processes

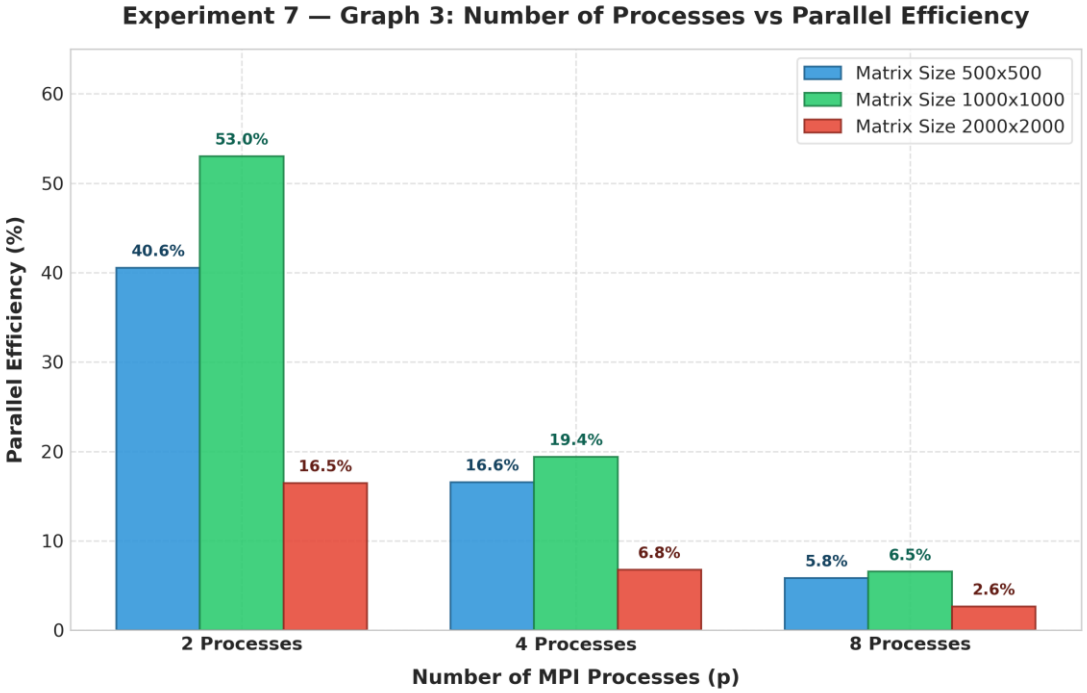
Required Graphs

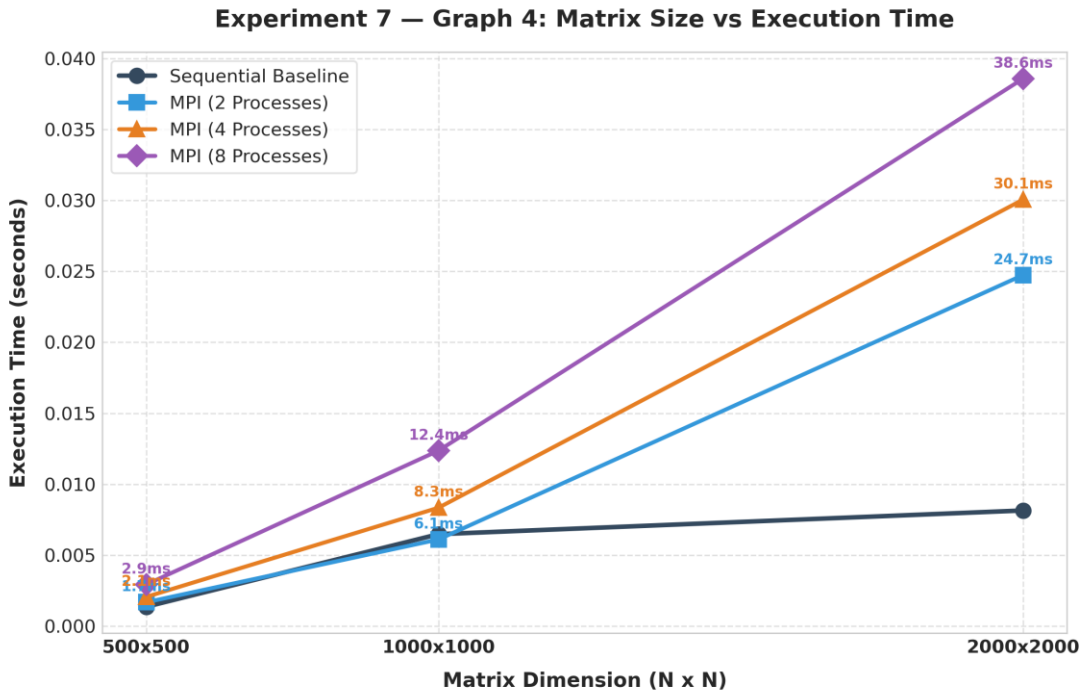
Prepare:

1. **Number of Processes vs Execution Time**



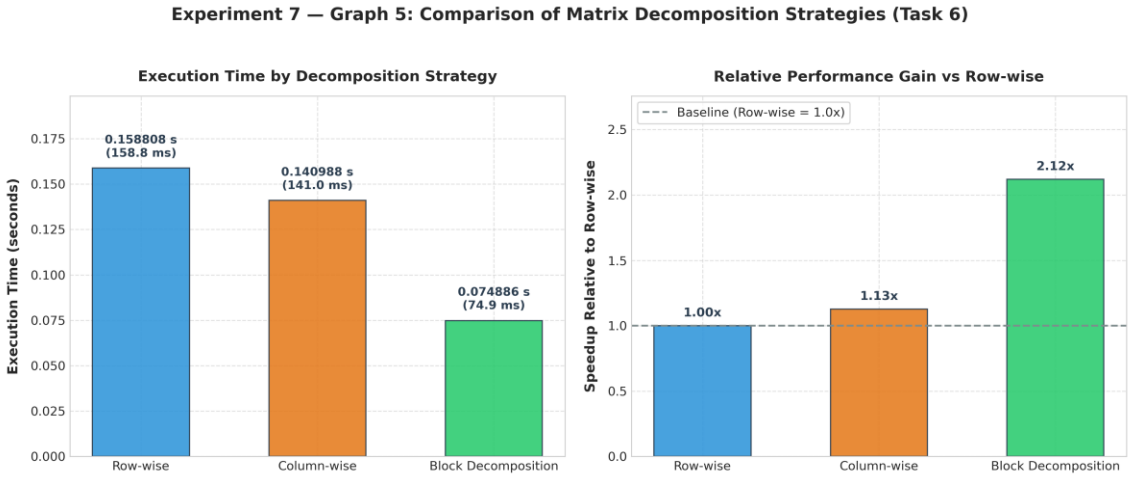

Experiment 7 — Graph 2: Number of Processes vs Speedup
Ideal Linear Speedup
Baseline (Speedup = 1.0)
MPI Matrix-Vector (N=500)
MPI Matrix-Vector (N=1000)
MPI Matrix-Vector (N=2000)
Speedup (T_seq / T_mpi)
Number of MPI Processes (p)
1.06x
0.81x
0.78x
0.52x
0.33x
0.27x
0.21x


Experiment 7 — Graph 3: Number of Processes vs Parallel Efficiency
Matrix Size 500x500
Matrix Size 1000x1000
Matrix Size 2000x2000
Parallel Efficiency (%)
Number of MPI Processes (p)
2 Processes
4 Processes
8 Processes
40.6%
53.0%
16.5%
16.6%
19.4%
6.8%
5.8%
6.5%
2.6%

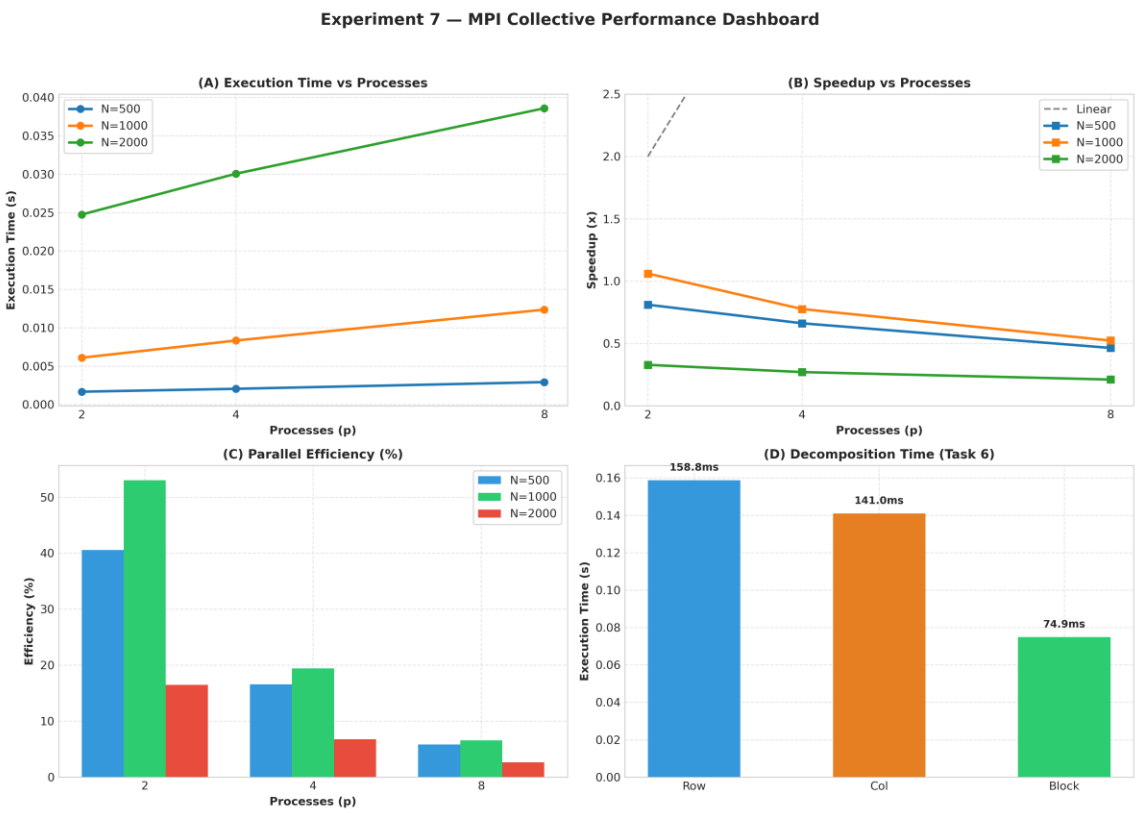


2. **Number of Processes vs Speedup**
3. **Number of Processes vs Efficiency**
4. **Matrix Size vs Execution Time**

Matrix Decomposition Strategies Comparison (Task 6)



MPI Collective Performance Dashboard



7. AI/HPC Case Study

Select one real-world application where distributed matrix computation or collective communication is used.

Suggested areas:

- Deep learning
- Computer vision
- Scientific computing
- Weather forecasting
- Computational fluid dynamics
- Medical imaging
- Recommendation systems
- Large-scale data analytics

- Molecular simulation

Write **300–500 words** covering:

1. Problem being solved.
2. Why the computation is computationally intensive.
3. How data can be distributed.
4. Role of MPI communication.
5. Possible decomposition strategy.
6. Benefits of parallel execution.
7. Communication or scalability challenges.

AI/HPC Case Study: Deep Learning

Application: Distributed Matrix Computation in Deep Learning

Deep learning is a major real-world application where distributed matrix computation and collective communication are widely used. Neural networks process large amounts of data, and most of their computation involves matrix multiplication. For example, in a fully connected layer, an input matrix is multiplied by a weight matrix to produce the output. Training modern deep learning models may involve millions or billions of parameters and large datasets, making the computation extremely intensive.

The main problem is to train a neural network efficiently on a large dataset within a reasonable amount of time. Matrix multiplication is computationally intensive because each output element requires many multiplication and addition operations. As the matrix size and number of layers increase, the total number of operations becomes very large.

The data can be distributed among multiple processors or computing nodes. For example, the rows of an input matrix can be divided among processes. Each process receives a subset of rows and performs matrix multiplication on its assigned portion. The weight matrix may be shared among all processes using a broadcast operation.

MPI plays an important role in coordinating these distributed computations. MPI_Scatter or MPI_Scatterv can distribute different portions of the input matrix to processes. MPI_Bcast can send a common weight matrix to all processes. After computation, MPI_Gather or MPI_Gatherv can collect the partial results into the final output matrix. Other collective operations such as MPI_Reduce can be used when partial results or gradients need to be combined.

A possible decomposition strategy is row-wise decomposition, where each process receives a different set of rows. For example, if a matrix has 500 rows and there are 8 processes, the rows can be divided approximately equally among the processes. Each process independently computes its assigned rows, reducing the workload of the main process.

Parallel execution can significantly reduce computation time and allow much larger models or datasets to be processed. It also makes better use of available CPU resources and can improve scalability when additional computing nodes are available.

However, communication introduces challenges. Distributing and collecting large matrices requires network bandwidth and time. Excessive communication can reduce the benefit of parallelization. Load imbalance can also occur when the data cannot be divided equally among processes. Therefore, an efficient distributed deep-learning system must balance computation, communication, memory usage, and workload distribution.

References:
MPI Forum – Collective Communication
Cornell MPI – Gather/Scatter
Distributed Deep Learning using MPI
MPI Matrix Multiplication – Scatter/Gather

8. Discussion Questions

1. What is collective communication?

   Collective communication is communication in which **multiple MPI processes participate together**. Examples are MPI_Bcast(), MPI_Scatter(), MPI_Gather(), and MPI_Reduce().

2. How does MPI_Bcast() work?

   - `MPI_Bcast()` sends the **same data from one root process to all other processes**.

     Example:

     ```
     MPI_Bcast(B, N*N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
     ```

   - Here, process 0 sends matrix `B` to every process.

3. Differentiate MPI_Scatter() and MPI_Gather().

MPI_Scatter()	**MPI_Gather()**
Distributes data from root to all processes	Collects data from all processes to root
One-to-many	Many-to-one
Used to divide input data	Used to combine output data

4. Differentiate MPI_Gather() and MPI_Reduce().

MPI_Gather()	**MPI_Reduce()**
Collects data from processes	Combines data using an operation
Root receives all individual results	Root receives one combined result
Example: collect rows of matrix C	Example: sum partial results
No mathematical operation required	Uses operations like MPI_SUM, MPI_MAX

5. Why is MPI_Bcast() useful in matrix multiplication?

   In matrix multiplication, all processes may need the **same matrix B**. Instead of sending B separately to every process, MPI_Bcast() efficiently distributes it from the root process to all processes.

6. Why is row-wise decomposition suitable for matrix-vector multiplication?

   Each output element of a matrix-vector multiplication depends on **one row of the matrix** and the complete vector. Therefore, different rows can be assigned to different processes and computed independently.

7. What are the limitations of row-wise decomposition?
   - Work may not be evenly distributed.
   - Each process may need access to the complete vector.
   - Large communication can occur when distributing data.
   - Memory usage can increase because common data may need to be stored by every process.
   - Performance can decrease when communication becomes significant.

8. What is block decomposition?

   Block decomposition divides a matrix into **smaller rectangular or square blocks** and assigns different blocks to different processes.

   For example:

```
A = | Block 1 | Block 2 |
    | Block 3 | Block 4 |
```

   Each process works on one or more blocks.

9. Why can block decomposition improve data locality?

   A process works repeatedly on a relatively small block of data that can fit better in **cache memory**. This reduces unnecessary memory accesses and can improve computational performance.

10. What is the purpose of MPI_Scatterv()?

    `MPI_Scatterv()` distributes **different amounts of data to different processes**.

    It is useful when data cannot be divided equally.

    For example, with **500 rows and 8 processes**, the rows can be distributed as:

```
63, 63, 63, 63, 62, 62, 62, 62
```

11. What happens when the number of data elements is not evenly divisible by the number of processes?

    Equal distribution using MPI_Scatter() may not be possible. We can use **MPI_Scatterv()** to distribute different amounts of data to different processes.

12. Why does execution time sometimes stop decreasing as the number of processes increases?

    Because increasing processes also increases **communication and synchronization overhead**. After a certain point, the communication overhead can offset the benefit of additional computation power.

13. How does communication overhead affect MPI scalability?

    Processes must exchange data and synchronize. As the number of processes increases, communication can become a larger portion of total execution time. Therefore, adding more processes may provide **smaller performance improvements**.

14. How does MPI differ from OpenMP in terms of memory architecture?

MPI	**OpenMP**
Generally uses distributed memory	Generally uses shared memory
Processes have separate address spaces	Threads share the same memory
Communication is explicit using MPI functions	Threads communicate through shared variables
Suitable for multiple machines/nodes	Mainly suitable for shared-memory systems

15. For a very large matrix, what factors should be considered when selecting a decomposition strategy?

    The following factors should be considered:

    - Matrix size
    - Number of processes
    - Available memory
    - Computational workload per process
    - Communication overhead
    - Data locality/cache usage
    - Whether data divides evenly
    - Network bandwidth and latency
    - Load balancing
    - Scalability

    For very large matrices, the goal is to achieve a good balance between computation communication, memory usage, and load distribution.

9. Reflection

Answer the following in your own words:

1. What was the most important concept you learned from this assignment?

   The most important concept I learned was **how MPI distributes a large computation among multiple processes**. In matrix multiplication, different processes can work on different rows of the matrix at the same time. I also understood how collective operations such as MPI_Scatter, MPI_Bcast, and MPI_Gather help processes share data and combine their results.

2. Which MPI collective operation did you find most useful and why?
   I found **MPI_Bcast()** most useful because the same matrix B is required by all processes during matrix multiplication. Instead of sending the matrix separately to each process, MPI_Bcast() sends it from the root process to all processes efficiently.

3. How is MPI matrix multiplication different from OpenMP matrix multiplication?
   In **MPI**, multiple processes have separate memory spaces and communicate with each other using MPI functions such as MPI_Scatter() and MPI_Gather(). In **OpenMP**, multiple threads share the same memory space and can directly access the matrices. Therefore, MPI requires explicit communication, while OpenMP mainly uses shared memory.

4. What happens when more MPI processes are added?
   When more MPI processes are added, the computation can be divided among more processes, so the execution time may initially decrease. However, after a certain point, communication and synchronization overhead increase. Because of this, adding more processes may give **smaller performance improvement** and can sometimes even increase the execution time.

5. Did increasing the number of processes always improve performance? Explain using your observations.

   No. Increasing the number of processes improved performance initially because the matrix computation was divided among more processes. However, after a certain point, the improvement became smaller because communication and synchronization overhead increased. Therefore, more processes do not always mean proportionally better performance.

6. Which decomposition strategy would you select for a large matrix and why?
   I would select **block decomposition** for a very large matrix. It divides the matrix into smaller blocks and distributes them among processes. This can improve **data locality**, reduce unnecessary data movement, and provide better load balancing compared with simple row-wise decomposition.

7. What communication bottlenecks did you observe?
   The main bottlenecks were **distributing the matrix, broadcasting matrix B, gathering the results, and synchronization between processes**. As the matrix size or number of processes increases, the amount of communication can become significant compared with the actual computation time.

8. How could the implementation be further optimized for an HPC cluster?
   The implementation could be optimized by:
   - Using **block or 2D decomposition** for large matrices.
   - Reducing unnecessary communication between processes.
   - Using optimized **MPI collective operations**.
   - Using **non-blocking communication** such as MPI_Isend() and MPI_Irecv() to overlap communication with computation.
   - Using optimized libraries such as **BLAS, OpenBLAS, or Intel MKL** for matrix operations.
   - Using hybrid **MPI + OpenMP** parallelism.
   - Improving **load balancing** between processes.
   - Taking advantage of the cluster's high-speed network and optimized MPI configuration.

10. Deliverables

Submit the following:

A. Programs

1. MPI Broadcast
2. MPI Scatter/Gather
3. MPI Reduce
4. MPI Matrix–Vector Multiplication
5. MPI Matrix–Matrix Multiplication
6. Decomposition comparison
7. MPI_Scatterv() implementation

B. Documentation

Include:

- Aim and objective

- Algorithm
- Source code
- Sample input/output
- Performance tables
- Graphs
- Speedup calculation
- Efficiency calculation
- Analysis of results
- AI/HPC case study
- Discussion-question answers
- Reflection
- Conclusion

C. Performance Evidence

Include execution results for different:

- Matrix sizes
- Number of MPI processes

- Decomposition strategies

11. Expected Learning Outcomes

After completing this assignment, you should be able to:

1. **Implement** MPI collective communication operations such as broadcast, scatter, gather and reduction.
2. **Apply** MPI to matrix-vector and matrix-matrix computations.
3. **Analyze** different data decomposition strategies for distributed-memory systems.
4. **Measure** execution time, speedup, efficiency and communication overhead.
5. **Evaluate** the effect of process count and data distribution on MPI performance.
6. **Relate** MPI-based parallel computation to real-world HPC and AI applications.